금 채굴하기

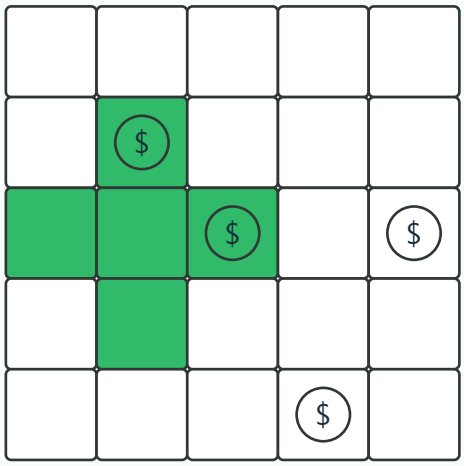
N × N크기의 이차원 영역에 파묻힌 금을 손해를 보지 않는 선에서 최대한 많이 채굴하려고 합니다. 채굴은 반드시 [그림 1, 2]와 같은 마름모 모양으로 단 한 번 할 수 있으며, 마름모 모양을 지키는 한 [그림 3]와 같이 이차원 영역을 벗어난 채굴도 가능하지만 이차원 영역 밖에 금은 존재하지 않습니다.

여기서 마름모 모양이란 특정 중심점을 기준으로 K번 이내로 상하좌우의 인접한 곳으로 이동하는 걸 반복했을 때 갈 수 있는 모든 영역이 색칠되어 있는 모양을 의미합니다. [그림 1]은 K가 1일 때의 마름모 모양이고, [그림 2]는 K가 2일 때 마름모 모양입니다. K가 0인 경우는 지정 한 곳에서만 채굴하는 것을 의미하며, 이 역시 올바른 마름모 모양이라고 할 수 있습니다. 올바르지 않은 마름모 모양을 이용해서는 채굴이 불가능합니다.

이 때 채굴에 드는 비용은 마름모 안의 격자 개수만큼 들어가며, 이는 $K^2 + (K + 1)^2$로 계산될 수 있습니다. 금 한 개의 가격이 M일 때, 손해를 보지 않으면서 채굴할 수 있는 가장 많은 금의 개수를 출력하는 코드를 작성해보세요. 단 한 개의 격자 안에는 최대 한 개의 금만 존재합니다.

codetree ▲

1 / 3



입력

첫 번째 줄에는 N과 M이 공백을 사이에 두고 주어지고,

두 번째 줄부터 (N + 1)번째 줄까지는 각 행에 금이 있는 경우 1, 없는 경우 0으로 입력이 공백을 사이에 두고 주어집니다.

제약 조건

- 1 ≤ N ≤ 20
- 1 ≤ M ≤ 10
- 각 격자값은 0 또는 1입니다.

출력

손해를 보지 않으면서 채굴할 수 있는 가장 많은 금의 개수를 출력합니다.

입출력 예시

예제 1

입력

```
5 5
0 0 0 0 0
0 1 0 0 0
0 0 1 0 1
0 0 0 0 0
0 0 0 1 0
```

출력

```
3
```

예제 설명

예제 1에서는 [그림 3]에서와 같이 (3, 3) 위치에 K가 2인 마름모를 그렸을 때, 채굴에 드는 비용은 13이고 금 3개의 가격은 15이므로 손해를 보지 않으면서 3개의 금을 얻을 수 있게 됩니다.

손해를 보지 않으면서 금 4개를 얻을 수 있는 방법은 존재하지 않습니다.

입출력 예시

예제 2

입력

```
3 2
0 1 0
1 0 1
0 0 0
```

출력

```
3
```

제한

Python3 1000 ms / 128 MB

모든 위치 확인하기

접근 1 메인 - 정답

Intuition

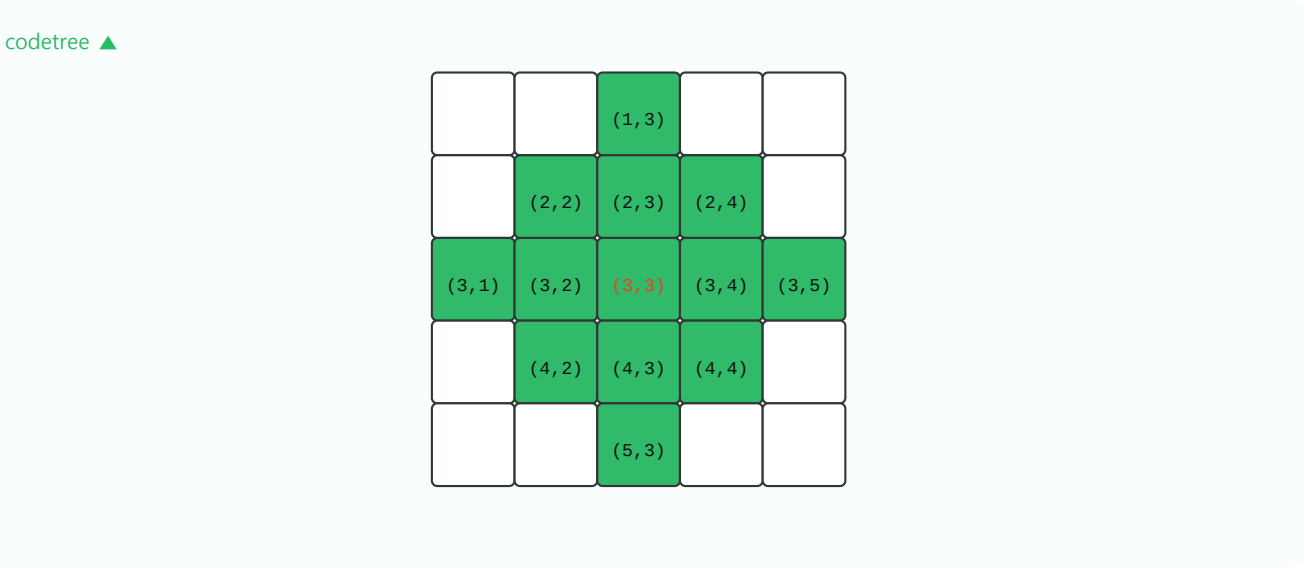
마름모의 중심의 위치와 K가 주어졌을 때 격자 내의 임의의 위치가 마름모에 포함되는지 여부는 마름모의 정의를 활용하여 쉽게 구할 수 있습니다. 이러한 특성을 활용하여 격자를 순회하며 마름모의 중심 후보를 정한 뒤, 가능한 K의 후보 각각에 대하여 몇 개의 금이 포함됐는지를 구할 수 있습니다.

Algorithm

마름모의 중심 위치와 K가 주어졌을 때 특정 위치가 마름모에 포함됐는지 어떻게 알 수 있을까요? 마름모의 정의에서 힌트를 얻을 수 있습니다. 마름모 모양은 중심점을 기준으로 K번 이내로 인접한 곳으로 이동하는 걸 반복했을 때 갈 수 있는 영역이므로, 다음 조건을 만족해야 합니다.

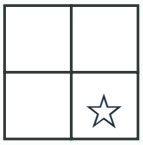
```
(중심점과의 x거리의 차이) + (중심점과의 y거리의 차이) ≤ K
```

즉, 마름모 중앙의 위치를 (a, b), 임의의 위치를 (c, d)라고 했을 때 |c - a| + |d - b| ≤ K 이면 해당 위치는 마름모에 포함된다고 할 수 있습니다. 이는 중심점이 (3, 3)이고 K = 2일 때 예인 아래의 그림에서도 확인할 수 있습니다.



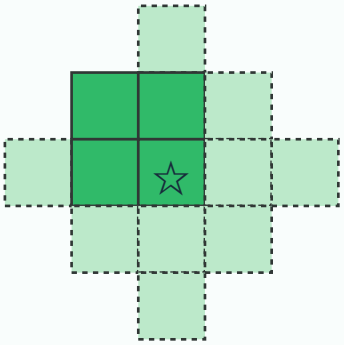
위와 같은 특성을 활용하여 격자를 순회하면서 각 위치가 마름모의 중심일 때 K를 0부터 가능한 범위까지 늘려가면서 손해를 보지 않고 채굴을 할 수 있는지 여부를 판별하면 됩니다. 이 때 가능한 K의 범위는 어떻게 정해질까요? 이럴 때는 최악의 경우를 산정하여 범위를 구하면 됩니다. 아래와 같이 N = 2이고 우측하단이 마름모의 중심일 때 모든 격자를 커버하려면 K = 2까지 커져야 합니다.

codetree ▲



K = 2일 때 아래의 그림과 같이 모든 영역을 커버할 수 있습니다. 이를 확장하여 생각해보면 격자 내에서 잡을 수 있는 가장 거리가 먼 두 점인 좌측상단과 우측 하단을 커버하기 위해서는 K = 2 × (N - 1)까지 커져야 함을 알 수 있습니다.

codetree ▲



전체 격자를 가능한 K 범위에 대하여 모두 탐색한 뒤 손해를 보지 않으면서 채굴할 수 있는 최대 금의 개수를 저장해주면 답을 구할 수 있습니다.

Code

```python
# 변수 선언 및 입력
n, m = tuple(map(int, input().split()))
grid = [
    list(map(int, input().split()))
    for _ in range(n)
]


# 주어진 k에 대하여 마름모의 넓이를 반환합니다.
def get_area(k):
    return k * k + (k + 1) * (k + 1)


# 주어진 k에 대하여 채굴 가능한 금의 개수를 반환합니다.
def get_num_of_gold(row, col, k):
    return sum([
        grid[i][j]
        for i in range(n)
        for j in range(n)
        if abs(row - i) + abs(col - j) <= k
    ])


max_gold = 0

# 격자의 각 위치가 마름모의 중앙일 때 채굴 가능한 금의 개수를 구합니다.
for row in range(n):
    for col in range(n):
        for k in range(2 * (n - 1) + 1):
            num_of_gold = get_num_of_gold(row, col, k)

            # 손해를 보지 않으면서 채굴할 수 있는 최대 금의 개수를 저장합니다.
            if num_of_gold * m >= get_area(k):
                max_gold = max(max_gold, num_of_gold)

print(max_gold)
```

Complexity

- Time Complexity: $O(N^5)$
- Space Complexity: $O(N^2)$

마름모 내부만 확인하기

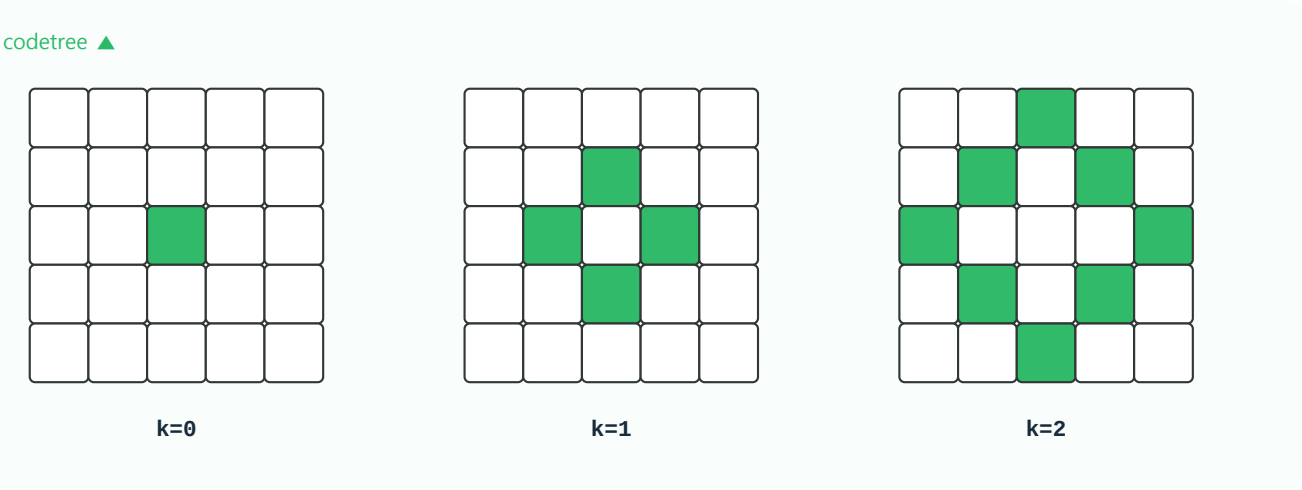
접근 2 정답

Intuition

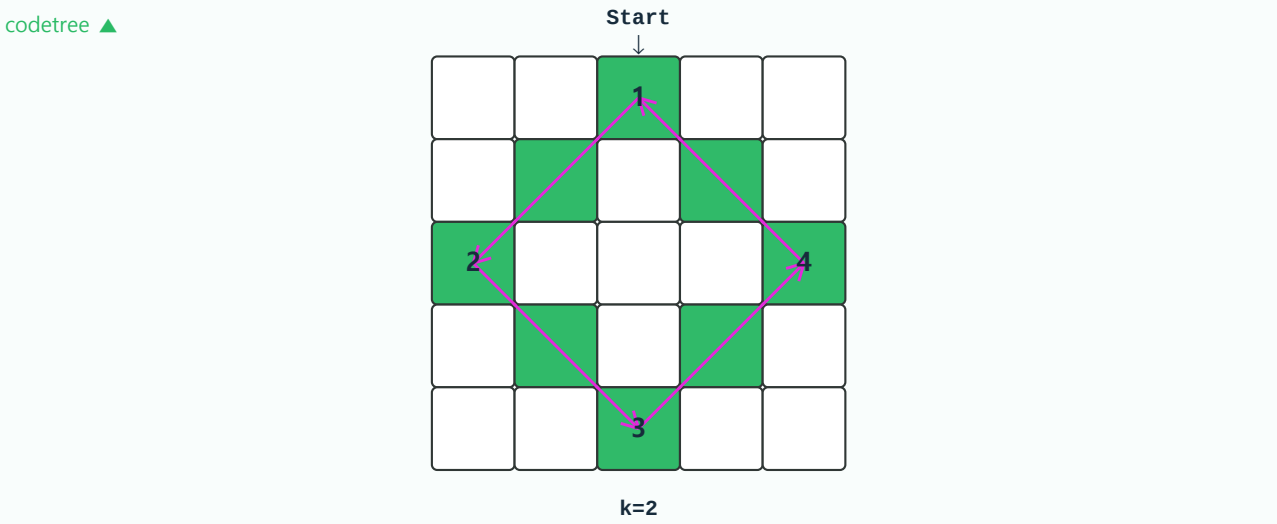
solution1에서는 매번 모든 격자를 다 확인하면서 금이 있는지 여부를 비효율적으로 탐색했습니다. 이를 마름모 내부의 영역만 확인하도록 수정하여 보다 효율적인 코드를 짤 수 있습니다. 단, 시간 복잡도는 동일합니다.

Algorithm

마름모의 중심 위치와 K가 주어졌을 때 마름모 내부를 어떤 방식으로 탐색할 수 있을까요? 예를 들어 K=2인 마름모 내부를 탐색하려면 아래의 그림과 같이 K=0, 1, 2일 때 각각의 가장자리 영역을 탐색한 결과를 합치면 됩니다. 단, K=0일 때는 별도의 가장자리가 없기 때문에 해당 위치에 금이 있는지 여부만 확인해주는 방식으로 예외 처리를 해주어야 합니다.



가장자리 영역을 탐색하기 위해서 우선 아래의 그림과 같이 시작점을 잡은 뒤, 네 방향에 대해서 탐색을 해줍니다. 이 때 각 방향에 대해 전진해야 하는 칸의 개수는 K개이며, dx dy 테크닉을 통해 보다 간결한 코드를 작성할 수 있습니다. (x, y)이 x행 y열을 나타낸다고 한다면, 1번 방향일 때는 x는 1증가, y는 1감소하게 되고, 2번 방향일 때는 x는 1증가, y는 1증가하게 됩니다. 이러한 방식으로 각 방향에 대한 x와 y의 변화량을 미리 구해두면 동일한 방향 내에서 다음 탐색할 위치를 쉽게 구할 수 있습니다.



위와 같은 방식으로 임의의 마름모 중심 위치와 K에 대하여 내부에 있는 금의 개수를 구할 수 있습니다. 전체 격자를 가능한 K 범위에 대하여 모두 탐색한 뒤 손해를 보지 않으면서 채굴할 수 있는 최대 금의 개수를 저장해주면 답을 구할 수 있습니다.

Code

```python
# 변수 선언 및 입력
n, m = tuple(map(int, input().split()))
grid = [
    list(map(int, input().split()))
    for _ in range(n)
]


# 주어진 k에 대하여 마름모의 넓이를 반환합니다.
def get_area(k):
    return k * k + (k + 1) * (k + 1)


# 주어진 좌표가 격자에 포함되는지 여부를 반환합니다.
def in_range(x, y):
    return 0 <= x and x < n and 0 <= y and y < n


# 주어진 k에 대하여 채굴 가능한 금의 개수를 반환합니다.
def get_num_of_gold(row, col, k):
    num_of_gold = 0
    # 방향에 따라 바뀌는 x와 y의 변화량을 정의합니다.
    dxs, dys = [1, 1, -1, -1], [-1, 1, 1, -1]

    num_of_gold += grid[row][col] # k = 0 일 때 처리

    for curr_k in range(1, k + 1):
        curr_x, curr_y = row - curr_k, col # 순회 시작점 설정
        for dx, dy in zip(dxs, dys):
            for step in range(curr_k):
                if in_range(curr_x, curr_y):
                    num_of_gold += grid[curr_x][curr_y]

                curr_x = curr_x + dx
                curr_y = curr_y + dy

    return num_of_gold


max_gold = 0

# 격자의 각 위치가 마름모의 중앙일 때 채굴 가능한 금의 개수를 구합니다.
for row in range(n):
    for col in range(n):
        for k in range(2 * (n - 1) + 1):
            num_of_gold = get_num_of_gold(row, col, k)

            # 손해를 보지 않으면서 채굴할 수 있는 최대 금의 개수를 저장합니다.
            if num_of_gold * m >= get_area(k):
                max_gold = max(max_gold, num_of_gold)

print(max_gold)
```

Complexity

- Time Complexity: $O(N^5)$
- Space Complexity: $O(N^2)$

중복되는 경우 제외하여 효율적으로 탐색하기

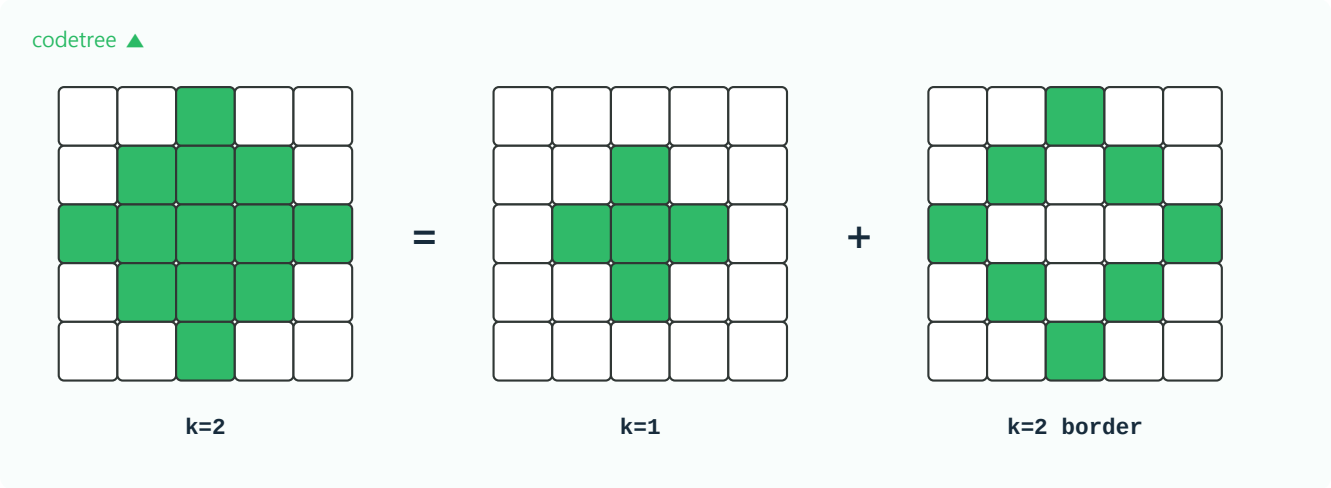
접근 3 정답

Intuition

solution2에서는 임의의 위치에서 가능한 K 범위에 대하여 모두 탐색할 때 매번 새로 마름모를 탐색하였습니다. 잘 생각해보면 K=a 일 때 내부에 있는 금의 개수는 (K=a-1 일 때의 금의 개수) + (K=a 일 때 그려지는 마름모의 가장자리에 있는 금의 개수)로 구할 수 있습니다. 이러한 특성을 활용하여 보다 효율적으로 탐색할 수 있습니다.

Algorithm

아래의 그림과 같이 k = 2일 때 k = 1일 때와 k = 2일 때의 가장자리의 합으로 구할 수 있습니다. 따라서 가능한 K 범위가 0 ~ 2 × (n - 1)에 대해서 탐색을 할 때 매번 새로 마름모 안의 영역을 탐색하는 것이 아니라 이전에 탐색했던 마름모 내부의 금 개수에 해당 마름모의 모서리에 있는 금의 개수만 더해주면 됩니다.



Code

```python
# 변수 선언 및 입력
n, m = tuple(map(int, input().split()))
grid = [
    list(map(int, input().split()))
    for _ in range(n)
]


def get_area(k):
    return k * k + (k + 1) * (k + 1)


def in_range(x, y):
    return 0 <= x and x < n and 0 <= y and y < n


# 주어진 k에 대하여 채굴 가능한 금의 개수를 반환합니다.
def get_num_of_gold_in_border(row, col, k):
    # 방향에 따라 바뀌는 x와 y의 변화량을 정의합니다.
    dxs, dys = [1, 1, -1, -1], [-1, 1, 1, -1]

    if k == 0:
        return grid[row][col]

    num_of_gold = 0

    curr_x, curr_y = row - k, col # 순회 시작점 설정
    for dx, dy in zip(dxs, dys):
        for step in range(k):
            if in_range(curr_x, curr_y):
                num_of_gold += grid[curr_x][curr_y]

            curr_x = curr_x + dx
            curr_y = curr_y + dy

    return num_of_gold


max_gold = 0

for row in range(n):
    for col in range(n):
        num_of_gold = 0
        for k in range(2 * (n - 1) + 1):
            # 이전 k까지 구한 금의 개수에
            # 해당 k의 모서리에 존재하는 금의 개수를 더해줍니다.
            num_of_gold += get_num_of_gold_in_border(row, col, k)

            # 손해를 보지 않으면서 채굴할 수 있는 최대 금의 개수를 저장합니다.
            if num_of_gold * m >= get_area(k):
                max_gold = max(max_gold, num_of_gold)

print(max_gold)
```

Complexity

- Time Complexity: $O(N^4)$
- Space Complexity: $O(N^2)$